

Clase 152 — Análisis de documentos maliciosos: macros y PDF

Parte: **6 — Análisis de malware** · Fuente: *Learning Malware Analysis* (Monnappa) y recursos de Didier Stevens ·  Duración estimada: **110 min** · Nivel: **Intermedio**

Objetivo

Analizar el vector de entrega más común en phishing: documentos ofimáticos con macros VBA y PDFs maliciosos. El alumno aprenderá a extraer y desofuscar macros, seguir la cadena de descarga/ejecución, examinar la estructura interna de un PDF y encontrar el JavaScript o los objetos que detonan la infección, todo sin abrir el documento en su aplicación real.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Explorar** la estructura OLE/OOXML de documentos Office sin abrirlos.
- 2. **Extraer y desofuscar** macros VBA y reconstruir la cadena de ejecución.
- 3. **Analizar** un PDF: objetos, streams, JavaScript y acciones automáticas.
- 4. **Reconocer** patrones de entrega (LOLBins, PowerShell, mshta).
- 5. **Extraer** IOCs (URLs, comandos, payloads) de forma segura.

Temas

#	Tema	Por qué importa
1	Formatos OLE vs OOXML	Determina la herramienta de extracción
2	Macros VBA y autoejecución	<code>AutoOpen</code> / <code>Document_Open</code> detonan
3	Desofuscación de VBA	El código real está escondido
4	Estructura de PDF	Objetos, streams, xref
5	JavaScript y acciones en PDF	<code>/OpenAction</code> , <code>/JS</code> , <code>/Launch</code>
6	Cadena de entrega (LOLBins)	mshta, powershell, wscript
7	Extracción de IOCs	Alimenta detección y bloqueo

Definiciones y características

- **Macro VBA:** código embebido en Office. Característica clave: **autoejecución** con `AutoOpen` .
- **OLE (Compound File):** contenedor binario de Office clásico (.doc/.xls). Característica clave: **streams internos**.
- **OOXML:** formato ZIP+XML moderno (.docx/.xlsm). Característica clave: **descomprimible**.

- **/OpenAction:** acción que un PDF ejecuta al abrirse. Característica clave: **detonación automática**.
- **LOLBin:** binario legítimo abusado (mshta, powershell). Característica clave: **evasión** por confianza.

Herramientas y preparación

- **oletools** (`olevba` , `oleid` , `oledump.py`) para Office.
- **peepdf**, **pdf-parser.py**, **pdfid.py** (Didier Stevens) para PDF.
- **CyberChef** para desofuscar cadenas.
- **REMnux** trae la mayoría preinstaladas.

 **Nota ética y de seguridad:** analiza los documentos con herramientas de línea de comandos **sin abrirlos** en Word/Adobe, y siempre en la VM aislada. Abrir el documento en su aplicación puede detonar la macro o el exploit. Trabaja solo con muestras autorizadas.

Laboratorio guiado

Documento Office con macros:

1. Identifica el tipo: `oleid documento.doc` . ¿Tiene macros?, ¿VBA?, ¿objetos embebidos?
2. Extrae y analiza la macro: `olevba documento.doc` . Localiza `AutoOpen` / `Document_Open` y marca las cadenas ofuscadas.
3. Desofusca: reconstruye concatenaciones, `Chr()` , base64 y XOR con CyberChef hasta obtener el comando real (típicamente un `powershell -enc ...`).
4. Decodifica el PowerShell (`-EncodedCommand` es base64 UTF-16LE) y extrae la URL de descarga y el payload de segunda etapa.

PDF malicioso:

5. Triaje: `pdfid.py doc.pdf` para contar `/JS` , `/OpenAction` , `/Launch` , `/EmbeddedFile` .
6. Extrae objetos: `pdf-parser.py --search JavaScript doc.pdf` y vuelca el stream del JS.
7. Analiza el JavaScript: identifica si descarga, lanza otro proceso o explota una vulnerabilidad del lector.
8. Consolida IOCs (URLs, comandos, hashes de payload) y mapea la cadena a ATT&CK (`T1566` , `T1059`).

Ejercicios

1. Explica la diferencia de análisis entre `.doc` (OLE) y `.docx/.xlsm` (OOXML).
2. Desofusca una macro que usa `Chr()` + concatenación.
3. Decodifica un `powershell -enc` y extrae la URL.
4. Interpreta la salida de `pdfid.py` y prioriza qué objeto revisar.
5. Extrae y explica un `/OpenAction` con `/Launch` .
6. Redacta 5 IOCs a partir de un documento analizado.

Reto verificable

Analiza un documento malicioso (Office o PDF) y entrega la **cadena de ejecución reconstruida** desde la apertura hasta el payload de segunda etapa, con la macro/JS desofuscado y los IOCs. **Criterio de**

aceptación: presentas el comando final desofuscado (p. ej. el PowerShell decodificado) y al menos una URL/IOC de la segunda etapa, sin haber abierto nunca el documento en su aplicación.

⚠ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Abrir el doc en Word para "verlo"	Detona la macro; usa solo oletools
oLevba sin resultados	Macro en objeto embebido; revisa oLedump.py por stream
PowerShell ilegible	Está en base64 UTF-16LE; decodifica con CyberChef
PDF "sin JS" pero sospechoso	Revisa /Launch , /EmbeddedFile y URIs, no solo /JS
Perder la segunda etapa	La URL responde en INetSim; captura el payload en el lab

? Preguntas frecuentes

? ¿Por qué no abrir el documento? Porque la macro o el exploit se ejecutan al abrir. Las herramientas de línea de comandos extraen el contenido sin detonarlo.

? ¿Siguen usándose macros? Menos desde que Office bloquea macros de Internet por defecto, pero persisten junto a otros vectores (HTML smuggling, LNK, contenedores).

? ¿Cómo decodifico un PowerShell ofuscado? Empieza por -EncodedCommand (base64 UTF-16LE) y luego deshaz capas (gzip, replace, format) con CyberChef.

🔗 Referencias

- Monnappa K. A., *Learning Malware Analysis*, cap. sobre documentos.
- oletools. <https://github.com/decalage2/oletools>
- Didier Stevens — PDF tools. <https://blog.didierstevens.com/programs/pdf-tools/>
- MITRE ATT&CK — Phishing (T1566). <https://attack.mitre.org/techniques/T1566/>

➡ Siguiente clase

Clase 153 - Analisis de malware en scripts: PowerShell y JavaScript